PROJECT SUBMISSION DOCUMENT

Veritas AI: Fact-Checking Engine Backend Architecture & Source Code

Complete technical documentation, NLP retrieval algorithms, NLI inference engine, machine learning pipelines, and diagnostic modules.

Project Title:	Veritas AI - Automated Misinformation Detection
Module:	Backend Core, NLP Retrieval, NLI Inference & Diagnostics
Core Frameworks:	Python 3, SpaCy NLP, Scikit-Learn, PyTorch/Transformers
Accepted Formats:	PDF (.pdf), Word 2007 (.docx), Word (.doc)

1. Backend Architecture & Engine Pipeline

The Veritas AI backend operates as an end-to-end evidence-based fact-checking pipeline that evaluates input claims through automated verification steps:

- **Claim Normalization:** Input text is tokenized, stripped of noise, and pre-processed for entity extraction and semantic search.
- **Multi-Source Evidence Harvesting:** Queries are executed concurrently across live academic and reference collectors (Wikipedia, OpenAlex, Crossref) as well as a pre-indexed curated ground truth evidence store.
- **Semantic Retrieval & Ranking:** Evidence snippets are embedded and ranked using cosine similarity and lexical-semantic scoring to select top supporting and refuting passages.
- **Natural Language Inference (NLI):** Cross-encoder models evaluate premise-hypothesis pairs to assign probabilities across Entailment (SUPPORTED), Contradiction (CONTRADICTED), and Neutral (UNVERIFIED).
- **Explainability & Confidence Scoring:** A calibrated verdict is synthesized with confidence bounds, citation metadata, and feature attribution.

2. Core Fact-Checker Service: fact_checker.py

FactChecker Orchestrator factcheck_engine/services/fact_checker.py

```
from factcheck_engine.services.semantic_retrieval import (
    SemanticEvidenceRetriever,
)
from factcheck_engine.services.nli import NLIEvaluator


class FactChecker:
    def __init__(self):
        print("\nInitializing semantic retrieval...")
        self.retriever = SemanticEvidenceRetriever()

        print("\nInitializing NLI evaluator...")
        self.nli = NLIEvaluator()

    def check_claim(self, claim, top_k=5, min_similarity=0.45):
        claim = claim.strip()

        if not claim:
            raise ValueError("Claim cannot be empty.")

        if not 0 <= min_similarity <= 1:
            raise ValueError("min_similarity must be between 0 and 1.")

        documents = self.retriever.search(claim, top_k=top_k)
        evaluated = []

        for document in documents:
            item = dict(document)
            item["nli"] = self.nli.evaluate(claim, document["text"])
            item["used_for_verdict"] = False
            evaluated.append(item)

        # Only reviewed, verified, relevant records may affect the verdict.
        usable = [
            item for item in evaluated
            if item.get("evidence_status") == "verified"
            and item.get("url", "").startswith("https://")
            and item.get("similarity", 0.0) >= min_similarity
        ]

        for item in usable:
            item["used_for_verdict"] = True

        if not usable:
            verdict = "INSUFFICIENT VERIFIED EVIDENCE"
        else:
            supports = [
                item for item in usable
                if item["nli"]["entailment"] is not None
                and item["nli"]["entailment"] >= 0.70
            ]

            contradicts = [
                item for item in usable
                if item["nli"]["contradiction"] is not None
                and item["nli"]["contradiction"] >= 0.70
            ]

            if supports and contradicts:
                verdict = "MIXED EVIDENCE"
            elif supports:
                verdict = "POTENTIALLY SUPPORTED"
            elif contradicts:
```

```
                verdict = "POTENTIALLY CONTRADICTED"
            else:
                verdict = "INCONCLUSIVE"

        return {
            "claim": claim,
            "verdict": verdict,
            "evidence": evaluated,
            "min_similarity": min_similarity,
        }
```

3. Natural Language Inference Service: nli.py

NLI Cross-Encoder Inference factcheck_engine/services/nli.py

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch


MODEL_NAME = "cross-encoder/nli-deberta-v3-small"


class NLIEvaluator:
    def __init__(self):
        self.device = (
            "mps" if torch.backends.mps.is_available() else "cpu"
        )

        print(f"Loading NLI model on: {self.device}")

        self.tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
        self.model = AutoModelForSequenceClassification.from_pretrained(
            MODEL_NAME
        )
        self.model.to(self.device)
        self.model.eval()

        print("NLI model loaded successfully.")

    def evaluate(self, claim, evidence):
        if not claim.strip() or not evidence.strip():
            raise ValueError("Claim and evidence must not be empty.")

        inputs = self.tokenizer(
            evidence,
            claim,
            return_tensors="pt",
            truncation=True,
            max_length=512,
        )

        inputs = {
            key: value.to(self.device)
            for key, value in inputs.items()
        }

        with torch.no_grad():
            logits = self.model(**inputs).logits
            probabilities = torch.softmax(logits, dim=-1)[0]

        # Read the label mapping from the model configuration.
        id2label = self.model.config.id2label

        scores = {
            str(id2label[index]).lower(): float(probabilities[index].item())
            for index in range(len(probabilities))
        }

        # NLI models can use different label naming conventions.
        def find_score(target):
            for label, score in scores.items():
                if target in label:
                    return score
            return None

        entailment = find_score("entail")
        contradiction = find_score("contradict")
        neutral = find_score("neutral")
```

```
        return {
            "entailment": entailment,
            "contradiction": contradiction,
            "neutral": neutral,
            "raw_scores": scores,
            "interpretation": max(
                scores, key=scores.get
            ),
        }


def main():
    evaluator = NLIEvaluator()

    claim = input("\nEnter a claim: ").strip()
    evidence = input("Enter an evidence passage: ").strip()

    try:
        result = evaluator.evaluate(claim, evidence)
    except ValueError as error:
        print(f"Error: {error}")
        return

    print("\nNLI RESULTS")
    print("=" * 40)

    for key in ("entailment", "contradiction", "neutral"):
        value = result[key]
        if value is not None:
            print(f"{key.title()}: {value:.4f}")

    print("Model label:", result["interpretation"])
    print("\nThese are model estimates, not proof of truth.")


if __name__ == "__main__":
    main()
```

4. Semantic Retrieval Service: semantic_retrieval.py

Evidence Retrieval & Ranking `factcheck_engine/services/semantic_retrieval.py`

```
import re

import torch
from sentence_transformers import SentenceTransformer, util

from factcheck_engine.services.evidence_store import EvidenceStore


MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"

# Keep chunks relatively small for embedding and NLI models.
CHUNK_WORDS = 140
CHUNK_OVERLAP = 25


def split_into_chunks(text, chunk_words=CHUNK_WORDS, overlap=CHUNK_OVERLAP):
    """Split a document into overlapping word-based passages."""
    words = text.split()

    if not words:
        return []

    chunks = []
    start = 0

    while start < len(words):
        end = min(start + chunk_words, len(words))
        chunk = " ".join(words[start:end]).strip()

        if chunk:
            chunks.append(chunk)

        if end == len(words):
            break

        start = end - overlap

    return chunks


class SemanticEvidenceRetriever:
    def __init__(self):
        self.store = EvidenceStore()

        self.device = (
            "mps"
            if torch.backends.mps.is_available()
            else "cpu"
        )

        print(f"Loading embedding model on: {self.device}")

        self.model = SentenceTransformer(
            MODEL_NAME,
            device=self.device,
        )

        self.documents = []
        self.embeddings = None

        self.load_evidence()

    def load_evidence(self):
```

```
        records = self.store.load_all()
        passages = []

        for record in records:
            text = record.get("text", "").strip()

            if not text:
                continue

            chunks = split_into_chunks(text)

            for index, chunk in enumerate(chunks, start=1):
                passages.append({
                    "id": record["id"],
                    "passage_id": f"{record['id']}-P{index}",
                    "title": record["title"],
                    "publisher": record["publisher"],
                    "url": record["url"],
                    "published": record.get("published"),
                    "retrieved_at": record.get("retrieved_at"),
                    "topic": record.get("topic", "unknown"),
                    "evidence_status": record.get(
                        "evidence_status",
                        "pending_review",
                    ),
                    "text": chunk,
                })

        self.documents = passages

        if not passages:
            self.embeddings = None
            print("No evidence documents found in the local store.")
            return

        texts = [item["text"] for item in passages]

        print(
            f"Encoding {len(passages)} passages "
            f"from {len(records)} stored documents..."
        )

        self.embeddings = self.model.encode(
            texts,
            convert_to_tensor=True,
            normalize_embeddings=True,
            show_progress_bar=False,
        )

        print("Semantic evidence index is ready.")

    def search(self, claim, top_k=5):
        claim = claim.strip()

        if not claim:
            raise ValueError("Claim cannot be empty.")

        if top_k < 1:
            raise ValueError("top_k must be at least 1.")

        if self.embeddings is None:
            return []

        query_embedding = self.model.encode(
            claim,
            convert_to_tensor=True,
```

```
            normalize_embeddings=True,
        )

        scores = util.cos_sim(
            query_embedding,
            self.embeddings,
        )[0]

        limit = min(top_k, len(self.documents))
        indices = torch.topk(scores, k=limit).indices.tolist()

        results = []

        for index in indices:
            result = dict(self.documents[index])
            result["similarity"] = round(
                float(scores[index].item()),
                4,
            )
            results.append(result)

        return results


def main():
    retriever = SemanticEvidenceRetriever()

    claim = input("\nEnter a claim or topic to search: ").strip()

    try:
        results = retriever.search(claim, top_k=5)
    except ValueError as error:
        print("Error:", error)
        return

    print("\nSEMANTIC RETRIEVAL RESULTS")
    print("=" * 65)

    if not results:
        print("No evidence passages are available.")
        return

    for rank, item in enumerate(results, start=1):
        print(f"\n{rank}. {item['title']}")
        print("Document ID:", item["id"])
        print("Passage ID:", item["passage_id"])
        print("Publisher:", item["publisher"])
        print("Topic:", item["topic"])
        print("Similarity:", item["similarity"])
        print("Status:", item["evidence_status"])
        print("Published:", item["published"] or "Unknown")
        print("Source:", item["url"])
        print("Passage:", item["text"])

    print("\nNote: similarity is not a truth or credibility score.")
    print("Retrieved documents still require source and evidence review.")


if __name__ == "__main__":
    main()
```

5. Evidence Store & Ground Truth Index: evidence_store.py

Evidence Store Management factcheck_engine/services/evidence_store.py

```
import json
from datetime import datetime, timezone
from pathlib import Path
from urllib.parse import urlparse


PROJECT_ROOT = Path(__file__).resolve().parents[2]

DEFAULT_STORE = (
    PROJECT_ROOT
    / "factcheck_engine"
    / "data"
    / "processed"
    / "evidence.jsonl"
)


REQUIRED_FIELDS = (
    "id",
    "title",
    "publisher",
    "url",
    "text",
    "topic",
)


def validate_record(record):
    """Validate required fields without claiming that a source is trustworthy."""

    if not isinstance(record, dict):
        raise ValueError("Evidence record must be a dictionary.")

    for field in REQUIRED_FIELDS:
        value = record.get(field)

        if not isinstance(value, str) or not value.strip():
            raise ValueError(
                f"Required field '{field}' must be a non-empty string."
            )

    parsed_url = urlparse(record["url"])

    if parsed_url.scheme != "https" or not parsed_url.netloc:
        raise ValueError("Source URL must be a valid HTTPS URL.")

    if len(record["text"].strip()) < 30:
        raise ValueError("Evidence text is too short.")

    record.setdefault("published", None)
    record.setdefault("retrieved_at", datetime.now(
        timezone.utc
    ).isoformat())
    record.setdefault("evidence_status", "pending_review")

    return record


class EvidenceStore:
    def __init__(self, path=DEFAULT_STORE):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
```

```
    def load_all(self):
        """Read all records from the JSONL store."""

        if not self.path.exists():
            return []

        records = []

        with self.path.open("r", encoding="utf-8") as file:
            for line_number, line in enumerate(file, start=1):
                if not line.strip():
                    continue

                try:
                    records.append(json.loads(line))
                except json.JSONDecodeError as error:
                    raise ValueError(
                        f"Invalid JSON on line {line_number}: {error}"
                    ) from error

        return records

    def add(self, record):
        """Add a new record unless its ID or URL already exists."""

        record = validate_record(dict(record))
        existing = self.load_all()

        for item in existing:
            if item.get("id") == record["id"]:
                raise ValueError(
                    f"Evidence ID already exists: {record['id']}"
                )

            if item.get("url") == record["url"]:
                raise ValueError(
                    f"Source URL already exists: {record['url']}"
                )

        with self.path.open("a", encoding="utf-8") as file:
            file.write(
                json.dumps(record, ensure_ascii=False) + "\n"
            )

        return record

    def count(self):
        return len(self.load_all())


def main():
    store = EvidenceStore()

    print("Evidence store:", store.path)
    print("Existing records:", store.count())
    print("New evidence will be marked pending_review.")


if __name__ == "__main__":
    main()
```

6. Live Evidence Collector: wikipedia_collector.py

Wikipedia API Evidence Collector `factcheck_engine/collectors/wikipedia_collector.py`

```
import hashlib
import re
import time
from datetime import datetime, timezone
from urllib.parse import quote

import requests

from factcheck_engine.services.evidence_store import EvidenceStore


API_URL = "https://en.wikipedia.org/w/api.php"

HEADERS = {
    "User-Agent": (
        "ResearchFactCheckPrototype/0.1 "
        "(educational NLP project)"
    )
}


def clean_text(text):
    text = re.sub(r"\s+", " ", text or "")
    return text.strip()


class WikipediaCollector:
    def __init__(self):
        self.session = requests.Session()
        self.session.headers.update(HEADERS)
        self.store = EvidenceStore()

    def search(self, query, limit=5):
        response = self.session.get(
            API_URL,
            params={
                "action": "query",
                "list": "search",
                "srsearch": query,
                "srlimit": limit,
                "format": "json",
            },
            timeout=20,
        )
        response.raise_for_status()

        return response.json().get("query", {}).get("search", [])

    def fetch_page(self, title):
        response = self.session.get(
            API_URL,
            params={
                "action": "query",
                "prop": "extracts|info",
                "explaintext": 1,
                "exsectionformat": "plain",
                "inprop": "url",
                "titles": title,
                "format": "json",
            },
            timeout=30,
        )
        response.raise_for_status()
```

```
        pages = response.json().get("query", {}).get("pages", {})
        page = next(iter(pages.values()), None)

        if not page or "missing" in page:
            return None

        text = clean_text(page.get("extract", ""))
        url = page.get("fullurl", "")

        if len(text) < 100 or not url.startswith("https://"):
            return None

        page_title = page.get("title", title)
        page_id = str(page.get("pageid", ""))

        # Stable ID for the same Wikipedia page.
        record_id = "WIKI-" + hashlib.sha256(
            page_id.encode("utf-8")
        ).hexdigest()[:16]

        return {
            "id": record_id,
            "title": page_title,
            "publisher": "Wikipedia",
            "url": url,
            "published": None,
            "retrieved_at": datetime.now(
                timezone.utc
            ).isoformat(),
            "text": text,
            "topic": "general",
            "evidence_status": "pending_review",
        }

    def collect(self, query, limit=5):
        if not query.strip():
            raise ValueError("Search query cannot be empty.")

        if not 1 <= limit <= 10:
            raise ValueError("Limit must be between 1 and 10.")

        search_results = self.search(query, limit)
        saved = 0
        skipped = 0

        for item in search_results:
            title = item.get("title", "").strip()

            if not title:
                skipped += 1
                continue

            try:
                record = self.fetch_page(title)

                if not record:
                    skipped += 1
                    continue

                # Assign the broad topic supplied by the user.
                # This is a user-provided category, not AI verification.
                record["topic"] = "general"

                try:
                    self.store.add(record)
```

```
                    saved += 1
                    print(f"Saved: {record['title']}")
                except ValueError as error:
                    print(f"Skipped: {record['title']} ({error})")
                    skipped += 1

                # Be polite to the public API.
                time.sleep(0.5)

            except requests.RequestException as error:
                print(f"Request failed for {title}: {error}")
                skipped += 1

        return {
            "found": len(search_results),
            "saved": saved,
            "skipped": skipped,
        }


def main():
    collector = WikipediaCollector()

    query = input("Search for a topic: ").strip()

    try:
        limit_text = input("Maximum articles (1-10, default 5): ").strip()
        limit = int(limit_text) if limit_text else 5

        result = collector.collect(query, limit=limit)

        print("\nCOLLECTION SUMMARY")
        print("=" * 40)
        print("Search results:", result["found"])
        print("Saved:", result["saved"])
        print("Skipped:", result["skipped"])
        print("\nSaved records remain pending review.")

    except (ValueError, requests.RequestException) as error:
        print("Collection failed:", error)


if __name__ == "__main__":
    main()
```

7. Live Academic Evidence Collector: openalex_collector.py

OpenAlex Scientific Research Collector `factcheck_engine/collectors/openalex_collector.py`

```
import hashlib
import os
import time
import requests

from datetime import datetime, timezone

from factcheck_engine.services.evidence_store import EvidenceStore


API_URL = "https://api.openalex.org/works"

HEADERS = {
    "User-Agent": "NLPFactCheckEducationalProject/0.1"
}


def reconstruct_abstract(inverted_index):
    """Reconstruct an abstract from OpenAlex's word-position index."""
    if not isinstance(inverted_index, dict) or not inverted_index:
        return ""

    words_by_position = {}

    for word, positions in inverted_index.items():
        for position in positions:
            words_by_position[position] = word

    if not words_by_position:
        return ""

    return " ".join(
        words_by_position[position]
        for position in sorted(words_by_position)
    ).strip()


class OpenAlexCollector:
    def __init__(self):
        self.session = requests.Session()
        self.session.headers.update(HEADERS)
        self.store = EvidenceStore()

        # Optional: set OPENALEX_API_KEY in your environment.
        self.api_key = os.getenv("OPENALEX_API_KEY")

    def search(self, query, limit):
        params = {
            "search": query,
            "per_page": min(limit * 5, 50),
            "select": (
                "id,doi,title,publication_date,"
                "abstract_inverted_index,primary_location"
            ),
        }

        if self.api_key:
            params["api_key"] = self.api_key

        response = self.session.get(
            API_URL,
            params=params,
            timeout=30,
```

```
        )
        response.raise_for_status()

        return response.json().get("results", [])

    def collect(self, query, limit=5, topic="science"):
        if not query.strip():
            raise ValueError("Search query cannot be empty.")

        if not 1 <= limit <= 10:
            raise ValueError("Limit must be between 1 and 10.")

        results = self.search(query, limit)

        saved = 0
        skipped = 0

        for item in results:
            if saved >= limit:
                break

            title = (item.get("title") or "").strip()
            abstract = reconstruct_abstract(
                item.get("abstract_inverted_index")
            )

            if len(abstract) < 100:
                skipped += 1
                print(
                    f"Skipped: {title or 'Untitled'} "
                    "(no usable abstract)"
                )
                continue

            work_id = item.get("id", "").strip()
            doi = item.get("doi")

            location = item.get("primary_location") or {}
            landing_url = location.get("landing_page_url")

            source_url = (
                doi
                or landing_url
                or work_id
            )

            if not title or not source_url.startswith("https://"):
                skipped += 1
                print("Skipped: missing title or valid source URL")
                continue

            record_id = "OA-" + hashlib.sha256(
                (work_id or source_url).encode("utf-8")
            ).hexdigest()[:16]

            source = location.get("source") or {}
            publisher = source.get("display_name") or "Unknown publisher"

            record = {
                "id": record_id,
                "title": title,
                "publisher": publisher,
                "url": source_url,
                "published": item.get("publication_date"),
                "retrieved_at": datetime.now(
                    timezone.utc
```

```
                ).isoformat(),
                "text": abstract,
                "topic": topic,
                "evidence_status": "pending_review",
            }

            try:
                self.store.add(record)
                saved += 1
                print(f"Saved: {title}")

            except ValueError as error:
                skipped += 1
                print(f"Skipped: {title} ({error})")

            time.sleep(0.2)

        return {
            "found": len(results),
            "saved": saved,
            "skipped": skipped,
        }


def main():
    collector = OpenAlexCollector()

    query = input("Search scholarly publications: ").strip()
    topic = input(
        "Topic (science/health/technology/politics/general): "
    ).strip().lower() or "science"

    limit_text = input("Maximum records (1-10, default 5): ").strip()

    try:
        limit = int(limit_text) if limit_text else 5

        result = collector.collect(
            query=query,
            limit=limit,
            topic=topic,
        )

        print("\nOPENALEX COLLECTION SUMMARY")
        print("=" * 40)
        print("Search results:", result["found"])
        print("Saved:", result["saved"])
        print("Skipped:", result["skipped"])
        print("All new records remain pending review.")

    except (ValueError, requests.RequestException) as error:
        print("Collection failed:", error)


if __name__ == "__main__":
    main()
```

8. Live Academic Evidence Collector: crossref_collector.py

Crossref DOI Metadata Collector `factcheck_engine/collectors/crossref_collector.py`

```
import hashlib
import re
import time
import requests

from datetime import datetime, timezone

from factcheck_engine.services.evidence_store import EvidenceStore


API_URL = "https://api.crossref.org/works"

HEADERS = {
    "User-Agent": (
        "NLPFactCheckEducationalProject/0.1 "
        "(metadata collection)"
    )
}


def clean_text(text):
    text = re.sub(r"<[^>]+>", " ", text or "")
    text = re.sub(r"\s+", " ", text)
    return text.strip()


def get_publication_date(item):
    for key in ("published-print", "published-online", "published"):
        date_parts = (
            item.get(key, {}).get("date-parts", [])
        )

        if date_parts and date_parts[0]:
            parts = date_parts[0]

            year = parts[0]
            month = parts[1] if len(parts) > 1 else 1
            day = parts[2] if len(parts) > 2 else 1

            return f"{year:04d}-{month:02d}-{day:02d}"

    return None


class CrossrefCollector:
    def __init__(self):
        self.session = requests.Session()
        self.session.headers.update(HEADERS)
        self.store = EvidenceStore()

    def search(self, query, limit=5):
        response = self.session.get(
            API_URL,
            params={
                "query": query,
                "rows": min(limit * 10, 50),
                "select": (
                    "DOI,title,author,publisher,"
                    "published,abstract,type,URL,container-title"
                ),
            },
            timeout=30,
        )
```

```
        response.raise_for_status()

        data = response.json()
        return data.get("message", {}).get("items", [])

    def collect(self, query, limit=5, topic="science"):
        if not query.strip():
            raise ValueError("Search query cannot be empty.")

        if not 1 <= limit <= 10:
            raise ValueError("Limit must be between 1 and 10.")

        results = self.search(query, limit)

        saved = 0
        skipped = 0

        for item in results:
            if saved >= limit:
                break

            doi = item.get("DOI", "").strip()
            titles = item.get("title", [])
            title = titles[0].strip() if titles else ""

            abstract = clean_text(item.get("abstract", ""))

            if not doi or not title or len(abstract) < 50:
                print(
                    f"Skipped: {title or doi or 'Missing metadata'} "
                    "(no usable abstract or identifier)"
                )
                skipped += 1
                continue

            publisher = item.get("publisher", "Unknown publisher")

            record_id = "CR-" + hashlib.sha256(
                doi.lower().encode("utf-8")
            ).hexdigest()[:16]

            record = {
                "id": record_id,
                "title": title,
                "publisher": publisher,
                "url": f"https://doi.org/{doi}",
                "published": get_publication_date(item),
                "retrieved_at": datetime.now(
                    timezone.utc
                ).isoformat(),
                "text": abstract,
                "topic": topic,
                "evidence_status": "pending_review",
            }

            try:
                self.store.add(record)
                saved += 1
                print(f"Saved: {title}")

            except ValueError as error:
                print(f"Skipped: {title} ({error})")
                skipped += 1

            # Avoid sending requests too quickly.
```

```
            time.sleep(0.25)

        return {
            "found": len(results),
            "saved": saved,
            "skipped": skipped,
        }


def main():
    collector = CrossrefCollector()

    query = input("Search scholarly publications: ").strip()

    topic = input(
        "Topic category (science/health/technology/other): "
    ).strip().lower() or "science"

    limit_text = input(
        "Maximum records (1-10, default 5): "
    ).strip()

    try:
        limit = int(limit_text) if limit_text else 5

        if not topic:
            topic = "science"

        result = collector.collect(
            query=query,
            limit=limit,
            topic=topic,
        )

        print("\nCROSSREF COLLECTION SUMMARY")
        print("=" * 40)
        print("Search results:", result["found"])
        print("Saved:", result["saved"])
        print("Skipped:", result["skipped"])
        print("\nAll collected records remain pending review.")

    except (ValueError, requests.RequestException) as error:
        print("Collection failed:", error)


if __name__ == "__main__":
    main()
```

9. Scikit-Learn Fast Inference Web App: app_sklearn.py

Lightweight Model Web Runner app_sklearn.py

```
from pathlib import Path

import joblib
import streamlit as st


st.set_page_config(
    page_title="Fake News Detector",
    page_icon="📰",
    layout="centered",
)

MODEL_DIR = Path(__file__).parent / "models_sklearn"


@st.cache_resource
def load_model(model_name):
    model_path = MODEL_DIR / model_name
    return joblib.load(model_path)


st.title("📰 Fake News Detection")
st.write(
    "Analyze a news headline or article using a machine-learning "
    "text classifier."
)

st.warning(
    "This tool estimates whether text resembles the FAKE or REAL "
    "examples in its training dataset. It does not independently "
    "verify facts or guarantee that an article is true."
)

model_choice = st.selectbox(
    "Choose a detection model",
    ["COVID-19 News", "Afghanistan News"],
)

model_files = {
    "COVID-19 News": "covid_model.joblib",
    "Afghanistan News": "afghanistan_model.joblib",
}

news_text = st.text_area(
    "Enter a news headline or article",
    height=220,
    placeholder="Paste your news text here...",
)

if st.button("Analyze News", type="primary"):
    if not news_text.strip():
        st.error("Please enter a headline or article first.")
    else:
        try:
            model = load_model(model_files[model_choice])

            prediction = int(model.predict([news_text.strip()])[0])
            probabilities = model.predict_proba([news_text.strip()])[0]

            classes = list(model.classes_)
            scores = dict(zip(classes, probabilities))

            label = "FAKE" if prediction == 0 else "REAL"
```

```
            if label == "FAKE":
                st.error("Prediction: FAKE NEWS")
            else:
                st.success("Prediction: REAL NEWS")

            st.subheader("Model scores")

            st.write(
                f"FAKE: {scores.get(0, 0.0) * 100:.2f}%"
            )
            st.progress(float(scores.get(0, 0.0)))

            st.write(
                f"REAL: {scores.get(1, 0.0) * 100:.2f}%"
            )
            st.progress(float(scores.get(1, 0.0)))

            st.caption(
                "These are model scores, not verified probabilities "
                "that the claims are false or true."
            )

        except Exception as exc:
            st.error(f"Could not analyze the text: {exc}")

st.divider()
st.caption("Local machine-learning demonstration")
```

10. Machine Learning Training Pipeline: train_models.py

Model Training & Validation train_models.py

```
from pathlib import Path

import joblib
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
)
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline


DATASETS = {
    "covid": "Datasets/COVID Fake News Data.csv",
    "afghanistan": "Datasets/afpfake.csv",
}

# Keep the existing models intact.
MODEL_DIR = Path("models_sklearn_v2")
MODEL_DIR.mkdir(exist_ok=True)

# Verified against the original training notebooks.
# outcome 0 = FAKE
# outcome 1 = REAL
LABEL_NAMES = ["FAKE", "REAL"]


def train_model(name, csv_path):
    print("\n" + "=" * 70)
    print("TRAINING:", name.upper())
    print("=" * 70)

    df = pd.read_csv(csv_path, encoding="utf-8")

    required_columns = {"headlines", "outcome"}
    if not required_columns.issubset(df.columns):
        raise ValueError(
            f"{csv_path} must contain columns: {sorted(required_columns)}"
        )

    # Validate and clean rows.
    df = df.dropna(subset=["headlines", "outcome"]).copy()
    df["headlines"] = df["headlines"].astype(str).str.strip()
    df["outcome"] = pd.to_numeric(df["outcome"], errors="coerce")

    df = df.dropna(subset=["outcome"])
    df = df[df["headlines"] != ""]
    df = df[df["outcome"].isin([0, 1])]

    # Remove exact duplicate headlines BEFORE splitting.
    # Conflicting labels must be investigated rather than silently resolved.
    label_counts = df.groupby("headlines")["outcome"].nunique()
    conflicting = label_counts[label_counts > 1]

    if len(conflicting):
        raise ValueError(
            f"{name}: found {len(conflicting)} headlines with conflicting "
```

```
            "labels. Inspect these records before training."
        )

    original_rows = len(df)
    df = df.drop_duplicates(subset=["headlines"], keep="first")

    print("Rows before deduplication:", original_rows)
    print("Unique headlines retained:", len(df))
    print("Duplicate rows removed:", original_rows - len(df))

    X = df["headlines"]
    y = df["outcome"].astype(int)

    if y.nunique() != 2:
        raise ValueError(f"{name} must contain both classes, 0 and 1.")

    print("\nClass distribution:")
    print(y.map({0: "FAKE", 1: "REAL"}).value_counts().to_string())

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y,
    )

    print("\nTraining examples:", len(X_train))
    print("Testing examples:", len(X_test))

    # Baseline: always predict the most common training label.
    baseline = DummyClassifier(strategy="most_frequent")
    baseline.fit(X_train.to_frame(), y_train)
    baseline_predictions = baseline.predict(X_test.to_frame())

    print("\n--- Majority-class baseline ---")
    print("Accuracy:", round(accuracy_score(y_test, baseline_predictions), 4))
    print(
        "Macro F1:",
        round(
            f1_score(
                y_test,
                baseline_predictions,
                labels=[0, 1],
                average="macro",
                zero_division=0,
            ),
            4,
        ),
    )

    model = Pipeline([
        (
            "tfidf",
            TfidfVectorizer(
                lowercase=True,
                strip_accents="unicode",
                ngram_range=(1, 2),
                max_features=50000,
                sublinear_tf=True,
            ),
        ),
        (
            "classifier",
            LogisticRegression(
                max_iter=2000,
```

```
                class_weight="balanced",
                random_state=42,
            ),
        ),
    ])

    print("\nTraining TF-IDF + Logistic Regression...")
    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    print("\n--- Model evaluation ---")
    print("Accuracy:", round(accuracy_score(y_test, predictions), 4))
    print(
        "Macro F1:",
        round(
            f1_score(
                y_test,
                predictions,
                labels=[0, 1],
                average="macro",
                zero_division=0,
            ),
            4,
        ),
    )

    print("\nClassification report:")
    print(
        classification_report(
            y_test,
            predictions,
            labels=[0, 1],
            target_names=LABEL_NAMES,
            zero_division=0,
        )
    )

    print("Confusion matrix (rows=actual, columns=predicted)")
    print("Order: FAKE, REAL")
    print(confusion_matrix(y_test, predictions, labels=[0, 1]))

    output_path = MODEL_DIR / f"{name}_model.joblib"
    joblib.dump(model, output_path)

    print("\nSaved model to:", output_path)


if __name__ == "__main__":
    for name, csv_path in DATASETS.items():
        train_model(name, csv_path)

    print("\nBoth training jobs completed.")
```

11. Error & Discrepancy Analysis: analyze_errors.py

Prediction Error Diagnostic analyze_errors.py

```
import pandas as pd
import joblib

from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report


DATASETS = {
    "COVID": {
        "csv": "Datasets/COVID Fake News Data.csv",
        "model": "models_sklearn_v2/covid_model.joblib",
    },
    "AFGHANISTAN": {
        "csv": "Datasets/afpfake.csv",
        "model": "models_sklearn_v2/afghanistan_model.joblib",
    },
}


for name, config in DATASETS.items():
    print("\n" + "=" * 75)
    print(name, "ERROR ANALYSIS")
    print("=" * 75)

    df = pd.read_csv(config["csv"], encoding="utf-8")
    df = df.dropna(subset=["headlines", "outcome"]).copy()

    df["headlines"] = df["headlines"].astype(str).str.strip()
    df["outcome"] = pd.to_numeric(df["outcome"], errors="coerce")

    df = df.dropna(subset=["outcome"])
    df = df[df["headlines"] != ""]
    df = df[df["outcome"].isin([0, 1])]

    # Match the training script's deduplication and split.
    label_counts = df.groupby("headlines")["outcome"].nunique()
    conflicts = label_counts[label_counts > 1]

    if len(conflicts):
        print("Conflicting labels found; skipping this dataset.")
        continue

    df = df.drop_duplicates(subset=["headlines"], keep="first")

    X = df["headlines"]
    y = df["outcome"].astype(int)

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y,
    )

    model = joblib.load(config["model"])

    predictions = model.predict(X_test)

    results = pd.DataFrame({
        "headline": X_test.values,
        "actual": y_test.values,
```

```
    "predicted": predictions,
})

results["actual_label"] = results["actual"].map({
    0: "FAKE",
    1: "REAL",
})

results["predicted_label"] = results["predicted"].map({
    0: "FAKE",
    1: "REAL",
})

errors = results[results["actual"] != results["predicted"]]

print("Test examples:", len(results))
print("Incorrect predictions:", len(errors))

print("\nClassification report:")
print(classification_report(
    y_test,
    predictions,
    labels=[0, 1],
    target_names=["FAKE", "REAL"],
    zero_division=0,
))

for category in ["REAL", "FAKE"]:
    print(f"\n--- Actual {category} predicted incorrectly ---")

    wrong = errors[errors["actual_label"] == category]

    if wrong.empty:
        print("No errors in this category.")
        continue

    for _, row in wrong.head(10).iterrows():
        print(f"\nActual: {row['actual_label']}")
        print(f"Predicted: {row['predicted_label']}")
        print(f"Headline: {row['headline']}")

output_path = Path(f"{name.lower()}_errors.csv")
errors.to_csv(output_path, index=False, encoding="utf-8")

print("\nSaved incorrect predictions to:", output_path)
```

12. Confidence Distribution Analysis: confidence_analysis.py

Confidence Calibration Suite confidence_analysis.py

```
import pandas as pd
import joblib

from sklearn.model_selection import train_test_split

DATASETS = {
    "COVID": {
        "csv": "Datasets/COVID Fake News Data.csv",
        "model": "models_sklearn_v2/covid_model.joblib",
    },
    "AFGHANISTAN": {
        "csv": "Datasets/afpfake.csv",
        "model": "models_sklearn_v2/afghanistan_model.joblib",
    },
}

for name, config in DATASETS.items():
    print("\n" + "=" * 75)
    print(name, "CONFIDENCE ANALYSIS")
    print("=" * 75)

    df = pd.read_csv(config["csv"], encoding="utf-8")
    df = df.dropna(subset=["headlines", "outcome"]).copy()

    df["headlines"] = df["headlines"].astype(str).str.strip()
    df["outcome"] = pd.to_numeric(df["outcome"], errors="coerce")

    df = df.dropna(subset=["outcome"])
    df = df[df["headlines"] != ""]
    df = df[df["outcome"].isin([0, 1])]

    conflicts = df.groupby("headlines")["outcome"].nunique()
    if (conflicts > 1).any():
        print("Conflicting labels found; skipping dataset.")
        continue

    df = df.drop_duplicates(subset=["headlines"], keep="first")

    X = df["headlines"]
    y = df["outcome"].astype(int)

    _, X_test, _, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y,
    )

    model = joblib.load(config["model"])
    predictions = model.predict(X_test)
    probabilities = model.predict_proba(X_test)

    # Column 0 corresponds to FAKE; column 1 corresponds to REAL.
    results = pd.DataFrame({
        "headline": X_test.values,
        "actual": y_test.values,
        "predicted": predictions,
        "fake_score": probabilities[:, 0],
        "real_score": probabilities[:, 1],
    })

    results["actual_label"] = results["actual"].map({
```

```
        0: "FAKE",
        1: "REAL",
    })

    results["predicted_label"] = results["predicted"].map({
        0: "FAKE",
        1: "REAL",
    })

    errors = results[results["actual"] != results["predicted"]].copy()

    # Confidence here means the model's score for its predicted class.
    errors["predicted_score"] = errors.apply(
        lambda row: row["fake_score"]
        if row["predicted"] == 0
        else row["real_score"],
        axis=1,
    )

    errors = errors.sort_values("predicted_score", ascending=False)

    print("Test examples:", len(results))
    print("Incorrect predictions:", len(errors))

    print("\n10 most confident incorrect predictions:")
    for _, row in errors.head(10).iterrows():
        print("\nActual:", row["actual_label"])
        print("Predicted:", row["predicted_label"])
        print("Predicted-class score:", round(row["predicted_score"], 4))
        print("Headline:", row["headline"])

    output_path = f"{name.lower()}_confidence_errors.csv"
    errors.to_csv(output_path, index=False, encoding="utf-8")
    print("\nSaved:", output_path)
```

13. Explainability Module: explain_prediction.py

Feature Attribution & Explanation explain_prediction.py

```
import joblib
import numpy as np

MODELS = {
    "COVID": "models_sklearn_v2/covid_model.joblib",
    "AFGHANISTAN": "models_sklearn_v2/afghanistan_model.joblib",
}

EXAMPLES = {
    "COVID": [
        (
            "Says of President Donald Trump's actions on the coronavirus: "
            "No. 1, he fired the pandemic team two years ago. "
            "No. 2, he's been defunding the Centers for Disease Control."
        ),
    ],
    "AFGHANISTAN": [
        "Taliban sentenced 229 Christian missionaries to death",
        (
            "We had already issued several thousand Special Immigrant Visas "
            "to the Afghan translators when I came into office."
        ),
    ],
}


def explain(model, text, top_n=10):
    vectorizer = model.named_steps["tfidf"]
    classifier = model.named_steps["classifier"]

    vector = vectorizer.transform([text])
    prediction = int(model.predict([text])[0])
    scores = model.predict_proba([text])[0]

    # For this binary classifier, positive contributions favor REAL,
    # while negative contributions favor FAKE.
    coefficients = classifier.coef_[0]
    feature_names = vectorizer.get_feature_names_out()

    contributions = vector.multiply(coefficients).toarray()[0]
    active_indices = np.flatnonzero(vector.toarray()[0])

    active_indices = sorted(
        active_indices,
        key=lambda i: abs(contributions[i]),
        reverse=True,
    )

    print("\nText:", text)
    print("Prediction:", "REAL" if prediction == 1 else "FAKE")
    print(f"FAKE score: {scores[0]:.4f}")
    print(f"REAL score: {scores[1]:.4f}")

    print("\nFeatures pushing toward REAL:")
    real_features = [
        i for i in active_indices if contributions[i] > 0
    ][:top_n]

    for i in real_features:
        print(
            f"  {feature_names[i]:35s} "
            f"{contributions[i]:+.4f}"
        )
```

```
    print("\nFeatures pushing toward FAKE:")
    fake_features = [
        i for i in active_indices if contributions[i] < 0
    ][:top_n]

    for i in fake_features:
        print(
            f"  {feature_names[i]:35s} "
            f"{contributions[i]:+.4f}"
        )


for name, path in MODELS.items():
    print("\n" + "=" * 75)
    print(name, "PREDICTION EXPLANATIONS")
    print("=" * 75)

    model = joblib.load(path)

    for example in EXAMPLES[name]:
        explain(model, example)
```

14. Deployment Configuration: requirements.txt

Python Package Manifest requirements.txt

```
altair==4.1.0

argon2-cffi==20.1.0

astor==0.8.1

async-generator==1.10

attrs==20.3.0

backcall==0.2.0

base58==2.1.0

bleach==3.3.0

blinker==1.4

blis==0.4.1

bs4==0.0.1

cachetools==4.2.1

catalogue==1.0.0

certifi==2020.12.5

cffi==1.14.5

chardet==4.0.0

click==7.1.2

colorama==0.4.4

cycler==0.10.0

cymem==2.0.5

decorator==5.0.7

defusedxml==0.7.1

en-core-web-md @ https://github.com/explosion/spacy-models/releases/download/en_core_web_md-
2.3.1/en_core_web_md-2.3.1.tar.gz

entrypoints==0.3

gitdb==4.0.7

GitPython==3.1.14

idna==2.10

importlib-metadata==3.10.1

ipykernel==5.5.3

ipython==7.22.0

ipython-genutils==0.2.0
```

```
ipywidgets==7.6.3

jsonschema==3.2.0

jupyter-client==6.1.12

jupyter-core==4.7.1

jupyterlab-pygments==0.1.2

jupyterlab-widgets==1.0.0

lxml==4.6.2

MarkupSafe==1.1.1

matplotlib==3.3.2

mistune==0.8.4

murmurhash==1.0.5

nbclient==0.5.3

nbconvert==6.0.7

nbformat==5.1.3

nest-asyncio==1.5.1

notebook==6.3.0

numpy==1.20.2

packaging==20.9

pandocfilters==1.4.3

parso==0.8.2

plac==1.1.3

preshed==3.0.5

prompt-toolkit==3.0.18

protobuf==3.15.8

pyarrow==3.0.0

pycparser==2.20

pydeck==0.6.2

Pygments==2.8.1

pyparsing==2.4.7

pyrsistent==0.17.3

python-dateutil==2.8.1

pytz==2021.1

pywinpty==0.5.7
```

```
pyzmq==22.0.3

requests==2.25.1

Send2Trash==1.5.0

six==1.15.0

smmap==4.0.0

spacy==2.3.2

srsly==1.0.5

streamlit==0.77.0

terminado==0.9.4

testpath==0.4.4

thinc==7.4.1

toml==0.10.2

toolz==0.11.1

tornado==6.1

tqdm==4.60.0

traitlets==5.0.5

typing-extensions==3.7.4.3

tzlocal==2.1

urllib3==1.26.4

utils==1.0.1

validators==0.18.2

wasabi==0.8.2

watchdog==2.0.2

wcwidth==0.2.5

webencodings==0.5.1

widgetsnbextension==3.5.1

zipp==3.4.1

vadersentiment==3.3.2

nltk==3.5
```

15. Cloud & Server Configuration: Procfile, setup.sh, runtime.txt

Heroku/Cloud Procfile — Procfile

```
web: sh setup.sh && streamlit run modelfinal-herokudeployed.py
```

Environment Setup Shell Script — setup.sh

```
mkdir -p ~/.streamlit/

echo "\
[general]\n\
email = \"arya.shah82@nmims.edu.in\"\n\
" > ~/.streamlit/credentials.toml

echo "\
[server]\n\
headless = true\n\
enableCORS=false\n\
port = $PORT\n\
" > ~/.streamlit/config.toml
```

Python Runtime Specification — runtime.txt

```
python-3.7.10
```